TERRAFORM COMPNENTS - FEATURES - CLOUD AUTHENTICATION / AUTHORIZATION

1. Provider — AWS ki Terraform ki madhya bridge
Terraform anedi oka **infrastructure automation tool**. Terraform ki AWS lo EC2 ela create cheyyali, S3 ela create cheyyali ani direct knowledge undadu. AWS API tho communication cheyyadaniki **AWS Provider** kavali. Nuvvu Terraform lo AWS Provider configure chesthe, Terraform AWS API ni use chesi AWS resources ni manage cheyyagalugutundi.
Important point enti ante **Provider itself EC2/VPC/S3 kaadu**. Provider ni oka **translator** laga anuko. Nuvvu Terraform ki "EC2 kavali" ani configuration lo cheptav; Terraform aa requirement ni AWS ki communicate cheyyadaniki Provider ni use chestundi. Oka AWS Provider tho EC2, VPC, S3, RDS, IAM ila chala different resources manage cheyyachu. **Provider = AWS tho communication cheyyadaniki; Resource = AWS lo em manage cheyyalo.**

Netflix example: Netflix tana AWS infrastructure ni Terraform tho manage chestundi anuko. Terraform ki AWS tho communicate avvadaniki **AWS Provider** use avtundi. Provider Netflix kosam EC2 create cheyyadu; instead Terraform requests ni AWS ki communicate cheyyadaniki bridge laga work chestundi.

Example — Netflix AWS
```
terraform {
 required_providers {
  aws = {
   source  = "hashicorp/aws"
   version = "~> 6.0"
  }
 }
}

provider "aws" {
 region = "us-east-1"
}
```

---

2. Resource — actual infrastructure ni represent chestundi
Resource Terraform lo chala important component, endukante nuvvu AWS lo create/manage cheyyalanukunna **actual infrastructure object** ni Resource represent chestundi. Example ki EC2 kavali ante EC2 resource, VPC kavali ante VPC resource, S3 bucket kavali ante S3 resource.
Nuvvu Resource declare chesinappudu Terraform ki basically oka instruction istunnav: **"AWS lo ee infrastructure undali."**
Terraform apply chesinappudu aa desired configuration ni AWS lo unna situation tho compare chesi required action chestundi. Kotha EC2 kavali ante create chestundi; existing EC2 lo configuration change unte update chestundi; configuration nunchi remove chesthe, situation batti resource ni destroy cheyyachu.
So **Resource anedi AWS service name matrame kaadu**; Terraform manage cheyyaboye specific infrastructure object ni represent chese block. **Provider AWS tho maatladithe, Resource AWS lo em undalo cheptundi.**

Netflix example: Netflix ki oka new application run cheyyadaniki AWS lo **EC2 servers** kavali anuko. Aa EC2 server Terraform lo Resource ga represent avtundi. Alage Netflix storage kosam S3, database kosam RDS, networking kosam VPC resources use cheyyachu.

Example — Netflix EC2
```
resource "aws_instance" "netflix_web" {
 ami         = "ami-xxxxxxxx"
 instance_type = "t3.medium"

 tags = {
  Name = "netflix-web-server"
 }
}
```

---

3. Variables — same Terraform code ni different situations lo use cheyyadaniki
Suppose oka company ki **Dev, QA, Prod** three environments unnayi. Infrastructure structure almost same, kani values different. Dev lo small EC2 kavachu, Prod lo large EC2 kavachu. Region, environment name, instance size, CIDR range ila values change avvachu. Prati environment kosam Terraform code motham copy chesi values change chesthe maintenance problem avtundi. Anduke **Variables** use chestam. Variable ante basically **"ee value fixed kaadu, tarvatha provide cheyyachu/change cheyyachu"** ani Terraform configuration ki cheppadam.
So Variables infrastructure ni create cheyyavu. **Resource ki kavalsina values ni provide chestayi.** Example ga Resource "EC2 create cheyyi" ani cheptundi; Variable "EC2 size t3.large" lanti value provide chestundi.

Netflix example: Netflix Dev environment lo small EC2 instances, Production lo powerful EC2 instances use cheyyalanukunte same Terraform setup ni reuse chesi **instance size value** matram change cheyyachu. Alage different AWS regions lo deploy cheyyalante region value kuda Variable ga provide cheyyachu.

Example — Netflix

```
variable "instance_type" {
  type    = string
  default = "t3.medium"
}

resource "aws_instance" "netflix_web" {
  ami           = "ami-xxxxxxxx"
  instance_type = var.instance_type
}
```

---

4. State — Terraform ki memory laga
Ippudu important question: Terraform oka EC2 create chesindi. Repu malli Terraform run chesinappudu, **"nenu already ee EC2 create chesanu"** ani ela telustundi?
Daanikosam **State**.
State lo Terraform manage chestunna infrastructure gurinchi information maintain chestundi. AWS lo create ayina resource ki AWS assign chesina IDs and Terraform ki aa resource ni identify/manage cheyyadaniki required information State lo track avutundi. Example ki Terraform already one EC2 manage chestundi. Nuvvu next time instance type change chesthe Terraform ki existing EC2 edho identify cheyyali. State aa mapping maintain cheyyadaniki important role play chestundi.
Simple ga:
Configuration = "Naaku em kavali?"
State = "Nenu already em manage chestunnanu?"
Terraform planning/apply process lo **configuration, state, and actual infrastructure information** ni use chesi required changes determine chestundi.

Netflix example: Netflix Terraform tho thousands of AWS resources manage chestundi anuko. Terraform already oka EC2 cluster manage chestunte, aa resources gurinchi information State lo track avtundi. Tarvatha Netflix configuration lo change chesthe Terraform existing infrastructure ni identify chesi required changes determine cheyyadaniki State ni use chestundi.

Example — Netflix
terraform.tfstate
↓
Netflix Web Server
↓
AWS EC2 Instance ID
↓
i-0123456789abcdef

---

5. Backend — Terraform State ni ekkada store cheyyalo
Backend State ni store/access cheyyadaniki Terraform ki mechanism provide chestundi. Small project lo State local machine lo undachu. Real company environment lo State ni central remote location lo maintain cheyyadam common.
AWS environment lo S3 backend use chesi Terraform State ni remote storage lo maintain cheyyachu. Team members and CI/CD pipeline same central State ni access cheyyagalru.

Netflix example: Netflix production Terraform State ni DevOps engineer laptop lo store cheyyakunda central S3 bucket lo store chestundi anuko. Team and CI/CD pipeline aa remote State ni use chestayi.

Example — S3 Backend
```
terraform {
  backend "s3" {
    bucket = "netflix-terraform-state"
    key    = "prod/terraform.tfstate"
    region = "us-east-1"
  }
}
```
Flow:
Terraform
↓
Backend
↓
S3
↓
terraform.tfstate
State = record
Backend = aa record ni store/manage cheyyadaniki mechanism

---

6. Module — Reusable Terraform code
Oka infrastructure setup ni multiple applications/environments lo reuse cheyyali ante Module use chestam. Module lo resources, variables, outputs lanti Terraform configuration ni organize chesi reusable package laga create chestam.

Dev, Stage, Prod anni same infrastructure architecture use chesthe same module ni multiple times call cheyyachu. Values environment batti change cheyyachu, kani actual infrastructure logic duplicate cheyyalsina avasaram ledu.
Netflix example: Netflix applications ki standard VPC + Subnet + Security Group setup undi anuko. Aa infrastructure ni netflix_network module ga create chesi Dev, Stage, Prod environments lo reuse cheyyachu.

Example — Module structure

```
modules/
└── netflix_app/
    ├── main.tf
    ├── variables.tf
    └── outputs.tf
```

Module call

```
module "netflix_app" {
 source = "./modules/netflix_app"

 environment   = "prod"
 instance_type = "t3.large"
}
```

Module main.tf

```
resource "aws_instance" "app" {
 instance_type = var.instance_type

 tags = {
   Name       = "netflix-app"
   Environment = var.environment
 }
}
```

Flow:

```
Netflix Module
    ↓
┌────┼────┐
↓    ↓    ↓
DEV STAGE PROD
```

---

7. Data Sources — Already unna AWS information ni read cheyyadam

Terraform dwara resource create cheyyakunda, AWS lo already existing resource information kavali ante Data Source use chestam. Data Source mainly read/query operation kosam.
Example ki AWS lo already VPC undi. Nuvvu aa VPC ni malli create cheyyakunda, existing VPC ID ni Terraform lo use cheyyali. Appudu Data Source existing VPC information ni fetch chestundi.
Netflix example: Netflix AWS account lo networking team already VPC create chesindi anuko. Application team aa existing VPC ni use cheyyali. Terraform Data Source tho VPC information read chesi application infrastructure ki reference ga use cheyyachu.

Example — Existing VPC

```
data "aws_vpc" "netflix" {
 filter {
   name   = "tag:Name"
   values = ["netflix-vpc"]
 }
}
```

Aa VPC ID ni resource lo use cheyyachu:

```
resource "aws_subnet" "netflix_app" {
 vpc_id    = data.aws_vpc.netflix.id
 cidr_block = "10.0.1.0/24"
}
```

Flow:

```
Existing AWS VPC
     ↓
  Data Source
     ↓
  VPC ID
     ↓
Terraform Resource
     ↓
New Subnet
```

Resource = create/manage
Data Source = existing information read

---

8. Provisioner — Resource operation tarvatha special command run cheyyadam

Terraform normal ga infrastructure create/manage cheyyadaniki use avtundi. Konni special situations lo resource create ayyaka additional command/script run cheyyali anipiste Provisioner use cheyyachu.

Common provisioners local-exec and remote-exec. local-exec Terraform run avtunna machine/CI runner lo command execute chestundi; remote-exec remote machine lo command execute cheyyagaladu.
Netflix example: Netflix EC2 server create ayyaka oka special initialization command run cheyyali anuko. Terraform resource creation tarvatha local-exec or remote-exec provisioner use cheyyachu. Kani possible ayithe cloud-init/user-data or dedicated configuration-management tools lanti alternatives ni consider cheyyadam common.

Example — local-exec

```
resource "aws_instance" "netflix_app" {
 ami        = "ami-0123456789"
 instance_type = "t3.micro"

 provisioner "local-exec" {
  command = "echo Netflix EC2 created"
 }
}
```

Flow:

Terraform
↓
Create EC2
↓
EC2 Created
↓
local-exec
↓
Extra Command

---

9. Service Principal / Cloud Identity — Terraform ki cloud authentication
Terraform AWS/Azure lanti cloud environments tho communicate cheyyalante identity + permissions kavali. Azure lo automation kosam Service Principal ane identity concept common. AWS lo similar purpose kosam IAM Roles/identities use chestaru.
Terraform CI/CD pipeline nunchi Azure resources create chestunnappudu human username/password tho login cheyyakunda, automation identity ki required permissions ichi authenticate cheyyachu.
Netflix example: Netflix Azure infrastructure ni Terraform tho manage chestundi anuko. CI/CD pipeline Terraform run chestunnappudu Azure Service Principal use chesi authenticate avtundi. Aa identity ki required Azure permissions matrame assign chestaru.

Example — Azure Service Principal authentication

```
provider "azurerm" {
 features {}

 subscription_id = var.subscription_id
 tenant_id      = var.tenant_id
 client_id      = var.client_id
 client_secret   = var.client_secret
}
```

Flow:

CI/CD Pipeline
↓
Service Principal
↓
Azure Authentication
↓
Azure API
↓
Azure Resources

Important: Service Principal Terraform core component kaadu. Idi mainly Azure authentication/identity concept. AWS side lo equivalent use case kosam IAM Roles lanti mechanisms use chestaru.

🔥 **Ippudu anni 9 kalipi oka real flow**
Nuvvu cheppav:
“AWS lo production application infrastructure kavali.”
First **IAM** Terraform ki AWS access identity + permissions istundi.
Then **Provider** Terraform ni AWS API tho communicate cheyyistundi.
Variables environment, region, instance size lanti values provide chestayi.
Already AWS lo unna VPC/subnet information kavali ante **Data Source** aa information fetch chestundi.
Standard infrastructure setup unte **Module** reusable configuration provide chestundi.
Actual infrastructure create/manage cheyyadaniki **Resources** use avtayi — EC2, VPC, S3, RDS etc.
Resource create ayyaka genuinely required special command unte **Provisioner** additional action chestundi.
Terraform manage chestunna infrastructure information ni **State** track chestundi.
Aa State ni local/remote ga ekkada and ela maintain cheyyalo **Backend** handle chestundi.

🧠 **Final ga ee 9 ni story laga gurthupettuko**

IAM → “Nuvvu evaru? Em permissions unnayi?”
Provider → “AWS tho nenu communication chestha.”
Variable → “Infrastructure ki values ivvu.”
Data Source → “Already unna AWS information teesukura.”
Module → “Ready-made Terraform setup reuse cheddam.”
Resource → “AWS lo ee infrastructure create/manage cheyyi.”
Provisioner → “Create ayyaka ee extra work cheyyi.”
State → “Nenu em manage chestunnano record pettuko.”
Backend → “Aa record ni ekkada/how maintain cheyyalo decide cheyyi.”
Most important connection:
Provider → AWS tho communication
Resource → AWS infrastructure
Variable → Resource ki values
Data Source → Existing AWS information
Module → Resources ni reusable setup ga organize cheyyadam
IAM → Access & permissions
State → Terraform memory
Backend → State storage/management
Provisioner → Extra action after resource operation.

TERRAFORM — Complete Classification

TERRAFORM
│
├── **1. CORE CONCEPTS**
│ │
│ ├── Provider
│ ├── Resource
│ ├── Variables
│ ├── State
│ ├── Backend
│ ├── Modules
│ ├── Data Sources
│ └── Provisioners
│
├── **2. TERRAFORM FEATURES**
│ │
│ └── Workspaces
│
├── **3. CLOUD AUTHENTICATION / AUTHORIZATION**
│ │
│ ├── AWS → IAM Roles / Users / Policies
│ └── Azure → Service Principal
│
├── **4. TERRAFORM + HASHICORP VAULT**
│ │
│ └── Secrets Management / Integration
│
└── **5. INFRASTRUCTURE ARCHITECTURE**
│
└── DR / Failover

Ippudu nee question lo unna 3 topics ekkada vastayo:

4. Terraform + HashiCorp Vault — Secrets ni securely use cheyyadam
Terraform infrastructure create/manage chestundi, kani infrastructure ki konni **sensitive values** kavali — example database password, API key, secret token. Aa secrets ni direct ga .tf files lo hardcode cheyyadam avoid cheyyali. **HashiCorp Vault** anedi ee sensitive secrets ni securely store/manage cheyyadaniki use ayye external tool. Terraform Vault tho integrate ayyi Vault nunchi required secret ni read chesi AWS resources/configuration lo use cheyyachu.
Important point enti ante **Vault Terraform component kaadu**. Vault oka separate **secrets management system**; Terraform daanitho integrate avtundi. Example ki Netflix application ki database password kavali anuko. Password ni main.tf lo password = "..." ani rayakunda Vault lo store chesi, Terraform Vault nunchi aa secret ni retrieve cheskovachu.
Netflix example: Netflix application AWS lo deploy avtundi anuko. Application ki RDS database password kavali. Aa password ni Terraform code lo direct ga pettakunda **HashiCorp Vault lo securely store** chestaru. Terraform Vault integration dwara aa password ni retrieve chesi required AWS configuration ki provide cheyyachu.

Example — Netflix + AWS + Vault
terraform {

```
 required_providers {
  aws = {
   source  = "hashicorp/aws"
   version = "~> 6.0"
  }

  vault = {
   source  = "hashicorp/vault"
  }
 }
}

provider "aws" {
 region = "us-east-1"
}

provider "vault" {
 address = "https://vault.netflix.example"
}
```

Vault nunchi secret read cheyyadaniki example:

```
data "vault_kv_secret_v2" "netflix_db" {
 mount = "secret"
 name  = "netflix/database"
}
```

Ikkada:

Vault
↓
Netflix DB Secret
↓
Terraform reads it
↓
AWS infrastructure/application

Simple ga:
Vault = secrets store chestundi
Terraform = infrastructure manage chestundi
Terraform + Vault = Terraform ki secure secrets provide cheyyadam

⚠️ Real production setup lo secrets Terraform State lo capture avvakunda design cheyyadam important, endukante Terraform state sensitive values ni contain cheyyagaladu.

---

5. Terraform Remote Backend — State ni remote location lo store cheyyadam

Terraform **State** lo Terraform manage chestunna infrastructure gurinchi information untundi. Small project ayithe State local machine lo undachu. Kani Netflix lanti company lo multiple developers, DevOps engineers, and CI/CD pipelines same Terraform infrastructure meeda work chestayi. Appudu State oka developer laptop lo matrame undadam practical kaadu. Anduke **Remote Backend** use chestam.

Remote Backend ante Terraform State ni **centralized remote storage** lo maintain cheyyadam. AWS environment lo S3 ni Terraform State storage kosam use cheyyachu. Team members and CI/CD pipeline same remote State ni use cheyyagalavu. State locking/concurrency control kuda backend setup and chosen backend capabilities batti configure cheyyachu.

Netflix example: Netflix production infrastructure ni 20 DevOps engineers manage chestunnaru anuko. State laptop lo unte andariki same State available undadu. Anduke Terraform State ni **AWS S3 bucket** lo central location ga store chestaru.

Example — Netflix AWS Remote Backend

```
terraform {
 backend "s3" {
  bucket = "netflix-terraform-state"
  key    = "production/terraform.tfstate"
  region = "us-east-1"
 }
}
```

Flow:

Netflix Developer
↓
Terraform
↓
Remote Backend
↓
AWS S3
↓
terraform.tfstate

Ikkada:

- bucket → State store ayye S3 bucket

- key → State file path/name
- region → S3 bucket region
  Important distinction:
  State = Terraform maintain chese infrastructure record.
  Backend = State ni ekkada/how maintain cheyyalo define chestundi.
  Remote Backend = State ni remote centralized location lo maintain cheyyadam.
  So **Remote Backend separate Terraform component kaadu; Backend topic lo important concept.**

---

6. Terraform Data Sources — Already unna AWS information ni read cheyyadam
Terraform lo **Resource** use chesthe generally AWS infrastructure ni create/manage chestam. Kani konni situations lo infrastructure already AWS lo exist avtundi, manam danini malli create cheyyakudadhu. Kani aa existing resource information Terraform ki kavali. Appudu **Data Source** use chestam.
Data Source **new infrastructure create cheyyadu**. AWS ni query/read chesi existing resource information Terraform configuration ki provide chestundi. Example ki Netflix AWS account lo already oka VPC undi. New application ki aa VPC use cheyyali, kani Terraform tho new VPC create cheyyakudadhu. Existing VPC details ni Data Source dwara fetch cheyyachu.
Netflix example: Netflix organization already vpc-123456 ane VPC ni AWS lo create chesindi anuko. New Netflix application deployment ki aa VPC ID/details kavali. Terraform data block use chesi existing VPC ni read chestundi.

Example — Netflix Existing AWS VPC

```
data "aws_vpc" "netflix_vpc" {
  id = "vpc-123456789"
}
```

Tarvatha aa existing VPC information ni resource lo use cheyyachu:

```
resource "aws_subnet" "netflix_subnet" {
  vpc_id     = data.aws_vpc.netflix_vpc.id
  cidr_block = "10.0.1.0/24"
}
```

Flow:

```
Existing Netflix VPC
        ↓
   Data Source
        ↓
VPC information
        ↓
   AWS Subnet Resource
```

Ikkada important difference:
Resource:
"New VPC/Subnet create cheyyi"

Data Source:
"Already unna VPC details naaku ivvu"
So:
Resource = Create / Manage
Data Source = Read / Fetch
Terraform Data Sources separate new component kaadu — mana original "Data Source" component ye.

---

important point enti ante Provider, Resource, Variables, State, Backend, Modules, Data Sources, Provisioners separate folders/files kaavu. Avi Terraform concepts; mostly .tf files lo define/use chestam.
Nee Netflix structure lo exact ga ekkada vastayo chudu:

```
netflix-infrastructure/
│
├── main.tf           ← Provider + Resources + Data Sources + Provisioners
├── variables.tf      ← Variables
├── terraform.tfvars  ← Variables ki actual values
├── outputs.tf        ← Outputs
│
├── terraform.tfstate ← State (terraform create chestundi)
│
└── modules/
    └── netflix/
        ├── main.tf     ← Module lo Resources
        ├── variables.tf ← Module inputs
        └── outputs.tf  ← Module outputs
```

Exact mapping

Component	Nee structure lo ekkada?	Simple meaning
Provider	main.tf	AWS tho Terraform communicate cheyyadaniki
Resource	main.tf / modules/netflix/main.tf	EC2, VPC, S3 etc. create/manage cheyyadaniki
Variables	variables.tf	Inputs declare cheyyadaniki
State	terraform.tfstate	Terraform manage chestunna infrastructure record
Backend	main.tf lo terraform { backend {} }	State ekkada store cheyyalo
Module	modules/netflix/	Reusable Terraform code
Data Source	main.tf / module main.tf	Already unna AWS information read cheyyadaniki
Provisioner	Resource block lopala	Resource create ayyaka special command run cheyyadaniki

Example ga actual code chudu

1. Provider → main.tf

```
provider "aws" {
  region = "us-east-1"
}
```

2. Resource → main.tf

```
resource "aws_instance" "netflix" {
  instance_type = "t3.micro"
}
```

3. Variable → variables.tf

```
variable "instance_type" {
  type = string
}
```

Value → terraform.tfvars

```
instance_type = "t3.large"
```

4. Backend → main.tf

```
terraform {
  backend "s3" {
    bucket = "netflix-terraform-state"
    key    = "prod/terraform.tfstate"
    region = "us-east-1"
  }
}
```

Meaning: **State file ni S3 lo store cheyyi.**

5. Data Source → main.tf

```
data "aws_vpc" "existing" {
  id = "vpc-123456"
}
```

Meaning: **Already unna VPC information teesuko; kotha VPC create cheyyaku.**

6. Provisioner → Resource lopala

```
resource "aws_instance" "netflix" {
  instance_type = "t3.micro"

  provisioner "local-exec" {
    command = "echo Netflix server created"
  }
}
```

Meaning: **EC2 create ayyaka ee command run cheyyi.**

7. Module → modules/netflix/

```
modules/
└── netflix/
    ├── main.tf
    ├── variables.tf
    └── outputs.tf
```

Root main.tf nunchi:

```
module "netflix" {
  source = "./modules/netflix"
}
```

Meaning: **modules/netflix lo unna reusable Terraform code ni use cheyyi.**

🧠 **Biggest confusion clear cheyyali ante**
Folder structure ≠ Terraform components list.
Nee folder:
main.tf
variables.tf
terraform.tfvars
outputs.tf
modules/
ivi **files/folders**.
Nee list:
Provider
Resource
Variables
State
Backend
Module
Data Source
Provisioner
ivi **Terraform concepts/features**.
Oka .tf file lo **multiple concepts** undachu.

For example main.tf lo:
main.tf
│
├── Backend
├── Provider
├── Data Source
├── Resource
└── Provisioner
And modules/netflix/main.tf lo:
modules/netflix/main.tf
│
└── Resources
Final flow
Netflix Terraform Project
│
├── main.tf
│ ├── Backend
│ ├── Provider
│ ├── Data Source
│ ├── Resource
│ └── Provisioner
│
├── variables.tf
│ └── Variables
│
├── terraform.tfvars
│ └── Variable values
│
├── outputs.tf
│ └── Outputs
│
├── terraform.tfstate
│ └── State
│
└── modules/
└── netflix/
├── main.tf
│ └── Resources
├── variables.tf
│ └── Module inputs
└── outputs.tf
└── Module outputs

One line lo:
👉 **Provider/Resource/Data Source/Provisioner/Backend = mostly .tf configuration lo untayi; Variables = variables.tf; State = terraform.tfstate; Module = modules/ folder.**

---

real-world Terraform lo inka konni important topics untayi.

Tarvatha important ga nerchukovalsinavi

9. **Outputs** → created infrastructure nunchi information bayataki ivvadam
10. **Locals** → repeated/derived values ni locally define cheyyadam
11. **count** → same resource multiple times create cheyyadam
12. **for_each** → multiple resources/items ni individually manage cheyyadam
13. **Conditional Expressions** → condition batti value/resource behavior change cheyyadam
14. **Functions** → strings, lists, maps etc. meeda operations cheyyadam
15. **Dynamic Blocks** → repeated nested configuration generate cheyyadam
16. **Workspaces** → same configuration ki separate Terraform states maintain cheyyadam
17. **Terraform Registry** → ready-made providers/modules use cheyyadam
18. **Dependency & References** → resources madhya relationship create cheyyadam
19. **Import** → already AWS lo unna infrastructure ni Terraform management loki teesukuravadam
20. **Lifecycle Rules** → resource create/update/delete behavior control cheyyadam

L

9. Outputs — Terraform create chesina resource information bayataki ivvadam

Outputs ante Terraform create/manage chesina infrastructure nunchi manaki kavalsina information ni bayataki chupinchadaniki use chestam. Resource create ayyaka daani **ID, IP address, DNS name** lanti values ni output ga tiskovachu. Outputs resource ni create cheyyavu; already unna resource information ni display/expose chestayi. Modules lo kuda output important, endukante module lopala create aina resource information ni module bayata use cheyyadaniki outputs use chestam.

Netflix example: Netflix AWS lo application kosam EC2 server create chesindi anuko. Terraform apply ayyaka aa EC2 **Instance ID and Public IP** manaki teliyali. Appudu Outputs use chestam. Terraform EC2 create chestundi, Outputs aa EC2 information ni bayataki istayi.

Example — Netflix AWS

```
resource "aws_instance" "netflix" {
 ami         = "ami-xxxxxxxx"
 instance_type = "t3.large"
}

output "netflix_instance_id" {
 value = aws_instance.netflix.id
}

output "netflix_public_ip" {
 value = aws_instance.netflix.public_ip
}
```

---

10. Locals — repeated values ni oka place lo define cheyyadam

Locals ante Terraform configuration lo repeated ga use cheyyalsina values ni oka place lo define chesi multiple resources lo reuse cheyyadaniki use chestam. Variables laga user nunchi input tiskovadaniki Locals primarily use cheyyavu. Usually Terraform configuration lopala common names, tags, calculated values ni maintain cheyyadaniki use chestam.

Netflix example: Netflix AWS lo EC2, S3, RDS anni resources ki Project = Netflix and Environment = Production tags same ga undali anuko. Prati resource lo same tags repeat cheyyakunda Locals lo okasari define chesi reuse chestam.

Example — Netflix AWS

```
locals {
 project    = "netflix"
 environment = "prod"

 common_tags = {
  Project    = local.project
  Environment = local.environment
  ManagedBy   = "Terraform"
 }
}

resource "aws_instance" "netflix" {
 instance_type = "t3.large"

 tags = local.common_tags
}
```

---

11. Count — same resource ni multiple times create cheyyadam

count ante oka resource block ni multiple instances ga create cheyyadaniki use chesedi. Suppose 5 EC2 servers kavali ante 5 separate resource blocks rayalsina avasaram ledu. count = 5 ani iste Terraform same resource configuration ni 5 instances ga create/manage chestundi. count.index tho each instance ki index number access cheyyachu.
Netflix example: Netflix application ki 3 identical web servers kavali anuko. Same EC2 configuration ni 3 times rayakunda count = 3 use chestam. Terraform 3 EC2 instances create chestundi.
Example — Netflix AWS

```
resource "aws_instance" "netflix" {
 count       = 3
 ami         = "ami-xxxxxxxx"
 instance_type = "t3.large"

 tags = {
  Name = "netflix-server-${count.index + 1}"
 }
}
```

Result:

```
netflix-server-1
netflix-server-2
netflix-server-3
```

---

12. for_each — different values tho multiple resources create cheyyadam
for_each kuda multiple resources create/manage cheyyadaniki use chestam. Kani count number/index based ga work chesthe, for_each **key/value** based ga work chestundi. Different servers ki different configuration kavali ante for_each chala useful.
Netflix example: Netflix application lo web, api, worker ane 3 server types unnayi anuko. Web ki t3.large, API ki t3.medium, Worker ki t3.small kavali. for_each tho prathi server ki separate value ivvachu.
Example — Netflix AWS

```
variable "servers" {
 default = {
  web    = "t3.large"
  api    = "t3.medium"
  worker = "t3.small"
 }
}

resource "aws_instance" "netflix" {
 for_each     = var.servers
 ami          = "ami-xxxxxxxx"
 instance_type = each.value

 tags = {
  Name = "netflix-${each.key}"
 }
}
```

Result:

```
netflix-web    → t3.large
netflix-api    → t3.medium
netflix-worker → t3.small
```

---

13. Conditional Expressions — condition batti value decide cheyyadam
Conditional Expression ante oka condition true/false batti Terraform different value select cheyyadaniki use chesedi. Syntax simple ga condition ? true_value : false_value. Dev, Stage, Prod environments ki same code use chestunnappudu environment batti configuration change cheyyadaniki useful.
Netflix example: Netflix Dev environment lo small server use cheyyali, Production lo bigger server use cheyyali anuko. Environment prod aithe t3.large, otherwise t3.micro select cheyyachu.
Example — Netflix AWS

```
variable "environment" {
 type = string
}

locals {
 instance_type = var.environment == "prod"
  ? "t3.large"
  : "t3.micro"
}

resource "aws_instance" "netflix" {
 ami          = "ami-xxxxxxxx"
 instance_type = local.instance_type
}
```

Meaning:

```
prod → t3.large
```

dev → t3.micro

---

14. Functions — Terraform values tho operations cheyyadam
Terraform lo strings, lists, maps, numbers lanti values meeda operations cheyyadaniki **built-in functions** untayi. Strings combine cheyyadam, list modify cheyyadam, values transform cheyyadam lanti tasks ki functions use chestam. Complex Terraform configurations lo dynamic values create cheyyadaniki functions useful.
Netflix example: Netflix application name, environment, component name combine chesi resource name create cheyyali anuko. Terraform join() function use chesi "netflix-prod-server" laga name create cheyyachu.
Example — Netflix AWS

```
locals {
 app       = "netflix"
 environment = "prod"

 server_name = join("-", [
  local.app,
  local.environment,
  "server"
 ])
}

output "server_name" {
 value = local.server_name
}
```

Result:

```
netflix-prod-server
```

---

15. Dynamic Blocks — repeated nested configuration automatically create cheyyadam
Dynamic Block ante resource lopala repeated nested blocks manually rayakunda, list/map nunchi dynamically generate cheyyadaniki use chesedi. Security Group ingress rules lanti configuration multiple times repeat avvalsina situations lo useful. Idi normal resource creation kanna konchem advanced Terraform feature.
Netflix example: Netflix application Security Group ki ports 80 and 443 rendu allow cheyyali anuko. Rendu ingress blocks manually rayakunda ports list nunchi dynamic ga generate cheyyachu.
Example — Netflix AWS

```
variable "ports" {
 default = [80, 443]
}

resource "aws_security_group" "netflix" {
 name = "netflix-sg"

 dynamic "ingress" {
  for_each = var.ports

  content {
   from_port   = ingress.value
   to_port     = ingress.value
   protocol    = "tcp"
   cidr_blocks = ["0.0.0.0/0"]
  }
 }
}
```

Meaning:

```
[80, 443]
  ↓
Dynamic Block
  ↓
Ingress 80
Ingress 443
```

---

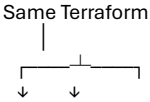
16. Workspaces — same Terraform code ki separate states maintain cheyyadam
Terraform Workspace ante same Terraform configuration ni use chestu separate Terraform states maintain cheyyadaniki use chese mechanism. Example ki same configuration ki dev and prod separate state maintain cheyyachu. Workspace **code ni separate cheyyadu**; mainly state ni separate chestundi.
Netflix example: Netflix same Terraform configuration ni Dev and Prod environments kosam use chestundi anuko. Dev resources oka state lo, Prod resources another state lo maintain cheyyadaniki workspaces use cheyyachu.
Example — Netflix

```
terraform workspace new dev
terraform workspace new prod

terraform workspace list
```

Flow:

Same Terraform Code



dev State prod State

17. Terraform Registry — ready-made Providers / Modules use cheyyadam

Terraform Registry ante Terraform providers and reusable modules dorike repository/catalog. Prati infrastructure component ni zero nunchi maname rayalsina avasaram ledu. Already suitable module unte documentation, inputs, outputs, versions check chesi reuse cheyyachu.

Netflix example: Netflix infrastructure kosam VPC setup kavali anuko. VPC module complete ga maname rayakunda Terraform Registry lo available AWS VPC module ni use cheyyachu.

Example — Netflix AWS

```
module "vpc" {
 source  = "terraform-aws-modules/vpc/aws"
 version = "6.0.1"

 name = "netflix-vpc"
 cidr = "10.0.0.0/16"
}
```

Flow:

Terraform Registry
↓
Ready-made VPC Module
↓
Netflix Infrastructure

---

18. Dependencies & References — resources madhya connection

Terraform infrastructure lo resources okadanipai okati depend avvachu. Example ki EC2 create cheyyadaniki mundu Security Group create avvali. Terraform lo oka resource attribute ni another resource nunchi reference chesthe Terraform aa dependency ni understand cheskuntundi.

Netflix example: Netflix EC2 ki Netflix Security Group attach cheyyali anuko. EC2 resource lo Security Group ID ni reference chestam. Appudu Terraform Security Group → EC2 dependency ni understand chestundi.

Example — Netflix AWS

```
resource "aws_security_group" "netflix" {
 name = "netflix-sg"
}

resource "aws_instance" "netflix" {
 ami         = "ami-xxxxxxxx"
 instance_type = "t3.large"

 vpc_security_group_ids = [
  aws_security_group.netflix.id
 ]
}
```

Flow:

Security Group
↓
ID
↓
EC2

---

19. Import — already existing AWS resource ni Terraform management loki teesukovadam

AWS lo oka resource already manually create chesi undachu. Danni delete chesi Terraform tho malli create cheyyalsina avasaram ledu. **Terraform Import** use chesi existing resource ni Terraform state lo associate chesi, tarvatha Terraform configuration tho manage cheyyachu.

Netflix example: Netflix AWS account lo already oka EC2 manually create chesi undi anuko. Ippudu DevOps team aa EC2 ni Terraform management loki teesukovali. Appudu aa EC2 ID ni import chestam.

Example — Netflix AWS

```
terraform import aws_instance.netflix i-123456789
```

Flow:

Existing AWS EC2
↓
terraform import
↓
Terraform State
↓
Terraform management

Important: Import chesina ventane complete Terraform configuration automatically ready ani assume cheyyakudadhu. Resource ki required configuration kuda properly define cheyyali.

---

20. Lifecycle Rules — resource create/update/delete behavior control cheyyadam
Lifecycle Rules resource ni Terraform create, update, destroy chesetappudu behavior ni control cheyyadaniki use chestam. Production resources accidentally delete kakunda prevent_destroy use cheyyachu. Resource replacement situation lo create_before_destroy use chesi new resource first create cheyyadaniki Terraform ki instruct cheyyachu.
Netflix example: Netflix Production Database chala important. Terraform mistake valla terraform destroy run ayina database delete kakudadhu anuko. prevent_destroy = true use cheyyachu.
Example — Netflix AWS

```
resource "aws_db_instance" "netflix" {
 engine = "mysql"

 lifecycle {
  prevent_destroy = true
 }
}
```

Meaning:
terraform destroy
↓
Netflix Database
↓
❌ Destroy prevent

🔥 **9–20 complete flow**

9. Outputs
↓
Resource information bayataki

10. Locals
↓
Common/calculated values

11. count
↓
Same resource multiple times

12. for_each
↓
Multiple resources with keys/values

13. Conditional
↓
Condition batti value

14. Functions
↓
Values meeda operations

15. Dynamic Blocks
↓
Repeated nested blocks

16. Workspaces
↓
Separate states

17. Registry
↓
Ready-made modules/providers

18. Dependencies
↓
Resources madhya relationship

19. Import
↓
Existing AWS → Terraform

20. Lifecycle
↓
Create/update/delete behavior

ADVANCED TERRAFORM — 1 TO 9

Ivi **basic components kaavu**. Real-world projects lo Terraform ni production level lo use chestappudu important ayye advanced topics.

---

1. Terraform + CI/CD — Terraform ni automatic deployment pipeline lo run cheyyadam
Real company lo Terraform code ni laptop nunchi manually terraform apply cheyyadam avoid chestaru. Terraform code GitHub/GitLab lanti Git repository lo untundi. Developer/DevOps engineer code change chesi Git ki push chesthe, **CI/CD pipeline** automatic ga Terraform commands run chestundi. First terraform fmt, validate, plan lanti checks run chesi, changes review/approval tarvatha terraform apply execute cheyyachu.
Netflix example: Netflix infrastructure Terraform code Git repository lo undi anuko. DevOps engineer production VPC lo change chesi Git push chestadu. CI/CD pipeline automatic ga Terraform plan generate chestundi. Approval tarvatha pipeline AWS lo aa infrastructure change ni apply chestundi.
Example — GitHub Actions

```
name: Terraform

on:
 pull_request:
 push:
  branches:
   - main

jobs:
 terraform:
  runs-on: ubuntu-latest

  steps:
   - uses: actions/checkout@v4

   - uses: hashicorp/setup-terraform@v3

   - run: terraform init

   - run: terraform validate

   - run: terraform plan
```

Flow

```
Developer
 ↓
Git Push
 ↓
GitHub
 ↓
CI/CD
 ↓
terraform plan
 ↓
Approval
 ↓
terraform apply
 ↓
AWS
```

---

2. Terraform + Vault — Secrets ni secure ga manage cheyyadam
Terraform ki konni situations lo database password, API token, certificate, secret key lanti sensitive values avasaram avvachu. Vatini main.tf lo direct ga rayadam or Git repository lo commit cheyyadam avoid cheyyali. **HashiCorp Vault** secrets ni secure ga store chesi Terraform required time lo retrieve cheyyadaniki use cheyyachu.

Netflix example: Netflix application database password Vault lo store chesindi anuko. Terraform ki aa password kavali. Terraform Vault ni query chesi password information teesukoni AWS RDS configuration lo use chestundi. Ila password ni Terraform code lo hardcode cheyyalsina avasaram undadu.
Example

```
provider "vault" {
 address = "https://vault.example.com"
}

data "vault_kv_secret_v2" "db" {
 mount = "secret"
 name  = "netflix/database"
}
```

```
resource "aws_db_instance" "netflix" {
 engine   = "mysql"
 username = data.vault_kv_secret_v2.db.data["username"]
 password = data.vault_kv_secret_v2.db.data["password"]
}
```

Flow

```
HashiCorp Vault
    ↓
Database Secret
    ↓
Terraform
    ↓
AWS RDS
```

---

3. Remote State & State Locking — Team lo same Terraform State ni safely use cheyyadam

Small project lo terraform.tfstate local laptop lo undachu. Kani real company lo multiple DevOps engineers Terraform use chestaru. State local ga unte team members madhya State synchronization problem ravachu. Anduke State ni remote backend lo central location lo store chestaru. Multiple people same time Terraform operation run chesi State ni conflict cheyyakunda **state locking/coordination** use chestaru.

Netflix example: Netflix production infrastructure ni 10 DevOps engineers manage chestunnaru anuko. State oka engineer laptop lo undakunda central remote storage lo untundi. Engineer A production deployment chestunnappudu Engineer B same State ni simultaneously modify cheyyakunda locking mechanism protect chestundi.

Example

```
terraform {
 backend "s3" {
  bucket = "netflix-terraform-state"
  key    = "prod/terraform.tfstate"
  region = "us-east-1"
 }
}
```

Flow

```
Developer A ──┐
              ↓
          Terraform
              ↓
       Remote State
              ↓
             S3
              ↑
       Locking / Coordination
              ↑
Developer B ──┘
```

Remember:

State = Terraform maintain chese infrastructure record
Backend = State ni ekkada maintain cheyyalo
Locking = Same State operations madhya conflicts prevent cheyyadam

---

4. Multi-Account AWS — Multiple AWS accounts ni Terraform tho manage cheyyadam

Real companies lo Dev, Stage, Production anni same AWS account lo undakapovachu. Security and isolation kosam separate AWS accounts maintain chestaru. Terraform lo **provider aliases** use chesi different AWS accounts ki different provider configurations create cheyyachu. Appudu correct resource ni correct AWS account lo deploy cheyyachu.

Netflix example: Netflix ki DEV, STAGE, PROD separate AWS accounts unnayi anuko. Terraform same configuration nunchi different provider aliases use chesi Dev resources Dev account lo, Production resources Prod account lo create cheyyagaladu.

Example

```
provider "aws" {
 alias  = "dev"
 region = "us-east-1"

 assume_role {
  role_arn = "arn:aws:iam::111111111111:role/TerraformRole"
 }
}

provider "aws" {
 alias  = "prod"
 region = "us-east-1"

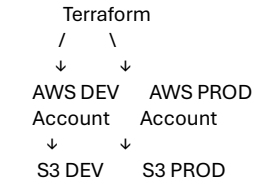
 assume_role {
  role_arn = "arn:aws:iam::222222222222:role/TerraformRole"
```

```
 }
}

resource "aws_s3_bucket" "dev" {
 provider = aws.dev
 bucket   = "netflix-dev-data"
}

resource "aws_s3_bucket" "prod" {
 provider = aws.prod
 bucket   = "netflix-prod-data"
}
```

Flow



---

5. Dev / Stage / Prod — Same infrastructure ni different environments lo manage cheyyadam

Real application ki usually **Dev, Stage, Prod** environments untayi. Architecture similar ga unna values different ga untayi. Dev lo small/cheap infrastructure, Stage lo testing infrastructure, Prod lo production-scale infrastructure undachu. Terraform Modules reuse chesi environment-specific values separate ga provide cheyyachu.

Netflix example: Netflix Dev environment ki 2 small EC2 servers kavachu. Stage ki bigger servers kavachu. Production ki multiple large EC2 servers kavachu. Infrastructure logic same module lo untundi, environment values matrame change avvachu.

Dev

```
module "netflix" {
 source = "../../modules/netflix"

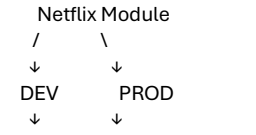
 environment   = "dev"
 instance_count = 2
 instance_type  = "t3.micro"
}
```

Prod

```
module "netflix" {
 source = "../../modules/netflix"

 environment   = "prod"
 instance_count = 10
 instance_type  = "t3.large"
}
```

Flow



---

6. DR / Failover — Disaster vachinappudu alternate infrastructure use cheyyadam

DR = Disaster Recovery. Primary infrastructure/region unavailable ayithe application ni recover cheyyadaniki alternative infrastructure maintain cheyyadam DR. Terraform ikkada DR infrastructure ni **code ga define cheyyadaniki and recreate cheyyadaniki** help chestundi. Terraform itself complete failover system kaadu; DNS, database replication, load balancing, application architecture lanti components tho kalisi DR solution work chestundi.

Netflix example: Netflix primary application us-east-1 region lo run avtundi anuko. us-west-2 lo required DR infrastructure Terraform tho define chestaru. Primary region lo major failure vaste architecture prakaram traffic DR environment ki shift avvachu.

Example

```
provider "aws" {
 alias  = "primary"
 region = "us-east-1"
}

provider "aws" {
 alias  = "dr"
 region = "us-west-2"
}

resource "aws_s3_bucket" "primary" {
```

```
 provider = aws.primary
 bucket   = "netflix-primary-data"
}

resource "aws_s3_bucket" "dr" {
 provider = aws.dr
 bucket   = "netflix-dr-data"
}
```

Flow

```
Primary Region
    ↓
Netflix Application
    ↓
Major Failure
    ↓
DR Region
    ↓
Backup Infrastructure
```

---

7. Security / Secrets Management — Infrastructure ni secure ga maintain cheyyadam

Terraform infrastructure create cheyyadam matrame kaadu; aa infrastructure ki **permissions and secrets** kuda safely manage cheyyali. AWS access keys, database passwords, API tokens lanti sensitive information ni Git lo plain text ga store cheyyakudadhu. IAM permissions ni required access ki limit cheyyachu, secrets ni AWS Secrets Manager, SSM Parameter Store, Vault lanti systems lo maintain cheyyachu.

Netflix example: Netflix production database password ni main.tf lo hardcode cheyyadu anuko. Secret management system lo password store chestaru. Terraform/application required permissions tho aa secret ni access chestayi.

Example — AWS Secrets Manager

```
data "aws_secretsmanager_secret" "db" {
 name = "netflix/database"
}

data "aws_secretsmanager_secret_version" "db" {
 secret_id = data.aws_secretsmanager_secret.db.id
}
```

Simple flow

```
Secret
 ↓
Secrets Manager / Vault
 ↓
Terraform / Application
 ↓
AWS Infrastructure
```

---

8. Terraform Testing — Terraform code correct ga work chestundo test cheyyadam

Real-world Terraform code production ki velladaniki mundu test cheyyali. Basic level lo terraform fmt, terraform validate, terraform plan use chestam. Advanced level lo Terraform testing framework use chesi modules/configuration expected behavior follow avtunnayo test cheyyachu. CI/CD pipeline lo tests automatic ga run cheyyadam valla code merge/apply mundu problems identify cheyyachu.

Netflix example: Netflix DevOps engineer VPC module lo change chesadu anuko. Git push ayyaka CI pipeline Terraform validation and tests run chestundi. Tests pass ayyaka plan/review stage ki velthundi.

Example

```
run "validate_network" {

 command = plan

 assert {
  condition    = aws_vpc.main.cidr_block == "10.0.0.0/16"
  error_message = "Unexpected VPC CIDR"
 }
}
```

Flow

```
Git Push
  ↓
terraform fmt
  ↓
terraform validate
  ↓
terraform test
  ↓
terraform plan
  ↓
```

Review

---

9. Module Design — Reusable Modules ni production standard lo design cheyyadam
Basic ga Module ante reusable Terraform code. **Module Design** ante aa reusable module ni real company lo clean, flexible, maintainable, configurable ga ela build cheyyali ane topic. Inputs ni variables.tf lo define chestam, actual resources main.tf lo create chestam, useful information outputs.tf dwara bayataki istam. Good module unnecessary hardcoding avoid chesi, validation, naming, tagging, outputs and versioning lanti things ni properly handle chestundi.
Netflix example: Netflix lo different application teams ki standard VPC architecture kavali anuko. DevOps team oka reusable VPC module design chestundi. Teams aa same module ni reuse chesi CIDR, environment, naming lanti values provide chestayi.
Example — VPC Module

```
modules/
└── vpc/
    ├── main.tf
    ├── variables.tf
    └── outputs.tf
```

main.tf

```
resource "aws_vpc" "main" {
  cidr_block = var.vpc_cidr

  tags = {
    Name        = var.name
    Environment = var.environment
  }
}
```

variables.tf

```
variable "vpc_cidr" {
  type = string
}

variable "name" {
  type = string
}

variable "environment" {
  type = string
}
```

Module ni use cheyyadam:

```
module "netflix_vpc" {
  source = "../../modules/vpc"

  vpc_cidr    = "10.0.0.0/16"
  name        = "netflix"
  environment = "prod"
}
```

Flow

```
Reusable VPC Module
        ↓
  ┌─────┼─────┐
  ↓     ↓     ↓
 DEV  STAGE  PROD
  ↓     ↓     ↓
 AWS   AWS   AWS
```

🔥 **Advanced 1 → 9**

```
1. Terraform + CI/CD
        ↓
2. Terraform + Vault
        ↓
3. Remote State + State Locking
        ↓
4. Multi-account AWS
        ↓
5. Dev / Stage / Prod
        ↓
6. DR / Failover
        ↓
7. Security / Secrets Management
        ↓
8. Terraform Testing
        ↓9. Module Design
```

Production / Senior-Level Pending Terraform Topics

1. State Surgery & Management Commands — State ni manually manage cheyyadaniki
Normal ga Terraform State ni Terraform automatic ga maintain chestundi. Kani production lo resource rename cheyyadam, module ki move cheyyadam, Terraform management nunchi resource remove cheyyadam lanti situations vastayi. Appudu State ni manually inspect/change cheyyalsi vastundi. Daanikosam terraform state commands use chestam.
state list → State lo unna resources chudataniki.
state show → Specific resource details chudataniki.
state mv → State mapping ni oka address nunchi another address ki move cheyyadaniki.
state rm → Resource ni Terraform management nunchi remove cheyyadaniki.

Netflix example: Netflix production lo existing EC2 resource name aws_instance.web nunchi aws_instance.netflix_web ga Terraform code lo rename chesindi anuko. Direct ga rename chesthe Terraform old resource destroy + new resource create cheyyalanukovachu. Existing EC2 ni destroy cheyyakunda State mapping ni move cheyyadaniki state mv use cheyyachu.
Example — Netflix

```
terraform state list

terraform state show aws_instance.netflix_web

terraform state mv \
  aws_instance.web \
  aws_instance.netflix_web

terraform state rm aws_instance.old_server
```

---

2. Drift Detection & Resource Replacement — Terraform code ki AWS reality ki difference detect cheyyadam
Terraform code lo manam oka configuration maintain chestam. Kani evaraina AWS Console loki velli manually resource configuration change chesthe, **Terraform configuration and actual AWS infrastructure madhya difference** create avvachu. Daanne **drift** antam.
terraform plan ಠ_ಠ Terraform infrastructure situation ni compare chesi changes ni identify cheyyagaladu. terraform plan -refresh-only mainly state/infrastructure information ni refresh chesi external changes ni inspect cheyyadaniki useful.
Konni situations lo oka particular resource ni intentionally recreate cheyyali. Appudu current Terraform versions lo terraform apply -replace="resource" use cheyyachu.

Netflix example: Netflix production EC2 ni evaro AWS Console nunchi manually change chesaru anuko. Terraform next plan lo aa infrastructure difference ni identify cheyyagaladu. Oka damaged EC2 ni complete ga recreate cheyyali ante aa resource ki -replace use cheyyachu.
Example — Netflix

```
terraform plan

terraform plan -refresh-only

terraform apply \
  -replace="aws_instance.netflix_web"
```

Simple ga:
Drift = “Code lo okati, AWS lo inkoti.”
Replace = “Ee particular resource ni destroy/recreate cheyyi.”

---

3. Policy as Code — Developer emaina infrastructure deploy cheyyakunda rules pettadam
Large company lo Terraform code technically valid unna kuda, company security rules violate cheyyachu. Example ki developer production lo unnecessary expensive GPU instance create cheyyakudadhu, S3 bucket public ga undakudadhu, encryption mandatory undali.
Daanikosam **Policy as Code** use chestam. Infrastructure deploy cheyyemundu predefined policies Terraform configuration/plans meeda enforce cheyyachu. HashiCorp ecosystem lo Sentinel undi; open-source environments lo OPA/Conftest lanti tools kuda use chestaru.
Netflix example: Netflix security team rule pettindi anuko: production S3 buckets public ga undakudadhu. Developer Terraform lo public S3 configuration create chesthe policy check deployment ni block cheyyagaladu.

Example — OPA/Conftest concept

```
package terraform

deny[msg] {
  resource := input.resource_changes[_]
  resource.type == "aws_s3_bucket"
  resource.change.after.acl == "public-read"

  msg := "Netflix production S3 bucket must not be public"
}
```

Meaning:
Terraform code → Policy check → Rule pass/fail → Deployment

4. DevSecOps & Security Scanning — Terraform code deploy cheyyemundu security check
Terraform code Git repository lo commit chese mundu security problems detect cheyyadam important. Example ki S3 public access, unrestricted SSH 0.0.0.0/0, missing encryption, insecure IAM permissions lanti problems accidentally code lo undachu. Checkov, Trivy, tfsec lanti tools Terraform/IaC configuration ni scan chesi security findings report cheyyagalavu. Ee checks ni local machine, pre-commit hooks, pull requests, CI/CD pipelines lo run cheyyachu.

Netflix example: Netflix developer Security Group lo SSH port 22 ni internet mothaniki open chesadu anuko. Terraform apply cheyyakundane CI pipeline security scanner aa configuration ni identify chesi PR ni fail cheyyachu.
Example — Netflix
checkov -d .

trivy config .

tfsec .
Flow:
Terraform Code
↓
Security Scanner
↓
Security Issues?
↙ ↘
YES NO
↓ ↓
FAIL CONTINUE

---

5. Infracost — Terraform change valla cost entha avuthundo mundhe telusukovadam
Infrastructure change apply cheyyakamundu **cost impact** telusukovadam production lo important. Oka developer EC2 size increase chesina, additional RDS create chesina, extra load balancers add chesina monthly cloud bill increase avvachu. **Infracost** Terraform configuration/plan ni analyze chesi estimated cloud cost difference chupinchagaladu. CI/CD pipeline lo Pull Request ki cost estimate/comment kuda integrate cheyyachu.

Netflix example: Netflix developer production EC2 ni t3.large nunchi expensive instance type ki change cheyyadu anuko. Terraform apply mundu Infracost aa infrastructure change valla estimated monthly cost difference chupinchagaladu.
Example — Netflix
infracost breakdown \
--path .

infracost diff \
--path .
Simple flow:
Terraform Change
↓
Infracost
↓
Current Cost vs New Cost
↓
PR Review
↓
Approve / Modify

---

6. Terragrunt — Large Terraform projects ni manage cheyyadaniki wrapper
Terraform itself powerful, kani large organizations lo same backend configuration, provider configuration, module inputs multiple environments/accounts lo repeat avvachu. **Terragrunt** Terraform paina work chese wrapper/tool, idi DRY configuration and dependency management lanti workflows ni simplify cheyyadaniki use chestaru.

Netflix example: Netflix ki dev, stage, prod environments tho paatu multiple AWS accounts unnayi anuko. Same Terraform modules ni different environments/accounts ki different inputs tho deploy cheyyali. Terragrunt common configuration ni centralize/reuse cheyyadaniki help cheyyachu.

Example — Netflix
terragrunt.hcl

terraform {
source = "../../modules/netflix-app"
}

inputs = {
environment = "production"
instance_type = "t3.large"
}
Simple ga:

Terraform = Infrastructure code
Terragrunt = Terraform configurations ni large-scale ga organize/manage cheyyadaniki wrapper

---

7. OpenTofu — Terraform-compatible open-source ecosystem option
OpenTofu anedi Terraform ecosystem nunchi vachina open-source Infrastructure as Code project. Terraform licensing changes tarvatha Linux Foundation umbrella kinda develop ayina project ga OpenTofu continue avtundi.
Conceptually Terraform configuration style ki close ga untundi, kani **Terraform and OpenTofu identical ani assume cheyyakudadhu**; specific versions/features/provider compatibility verify cheyyali.

Netflix example: Netflix organization ki open-source licensing requirements important ani assume cheddam. Terraform-compatible workflow ki alternative evaluate chestunnappudu OpenTofu ni consider cheyyachu.

Example — OpenTofu
```
terraform {
 required_providers {
  aws = {
   source  = "hashicorp/aws"
   version = "~> 6.0"
  }
 }
}

provider "aws" {
 region = "us-east-1"
}

resource "aws_instance" "netflix_web" {
 instance_type = "t3.micro"
}
```
Commands conceptually:
```
tofu init
tofu plan
tofu apply
```

---

8. Terraform State Import — Existing infrastructure ni Terraform management loki teesukovadam
Company AWS infrastructure already manually create chesi undachu. Terraform start chesinappudu aa resources anni malli create cheyyadam correct kaadu. Existing resource ni Terraform State lo associate cheyyadaniki **import** functionality use cheyyachu.
Important point: import existing infrastructure ni Terraform management loki bring chestundi; **complete Terraform configuration automatically design chestundi ani assume cheyyakudadhu.** Configuration ni properly define cheyyali.

Netflix example: Netflix already AWS lo manually create chesina production VPC undi. Ippudu DevOps team Terraform management loki migrate cheyyali anuko. Existing VPC ni Terraform state loki import cheyyachu.

Example — Netflix
```
terraform import \
 aws_vpc.netflix_vpc \
 vpc-0123456789abcdef
```
Flow:
```
Existing AWS VPC
    ↓
terraform import
    ↓
Terraform State
    ↓
Terraform manages it
```

---

9. Terraform Testing — Terraform code correct ga work avutundo test cheyyadam
Production infrastructure code ni blindly apply cheyyakunda tests run cheyyadam useful. Terraform native testing kosam .tftest.hcl files use cheyyachu. Tests configuration expected behavior ni verify cheyyadaniki use avvachu.
Netflix example: Netflix module production environment kosam VPC create chestundi anuko. Test lo VPC CIDR correct ga unda, expected resources create avuthunnaya lanti behavior verify cheyyachu.

Example — Netflix
```
# tests/netflix.tftest.hcl

run "validate_vpc" {

 command = plan

 assert {
  condition    = aws_vpc.netflix.cidr_block == "10.0.0.0/16"
```

```
  error_message = "Netflix VPC CIDR is incorrect"
 }
}
```

Run:

```
terraform test
```

---

10. Advanced Module Design — Production-quality reusable modules

Simple module create cheyyadam kanna production module design important. Good module lo clear inputs, outputs, validation, sensible defaults, naming conventions, documentation and versioning untayi.

Module ni specific application kosam hardcode cheyyakunda **reusable infrastructure building block** laga design cheyyali.

Netflix example: Netflix organization lo hundreds of applications same EC2 pattern use chestunnayi anuko. Oka reusable EC2 module create chesi application name, instance type, environment, subnet ID lanti values inputs ga teesukoni common standards enforce cheyyachu.

Example — Netflix Module

```
module "netflix_ec2" {
 source = "./modules/ec2"

 app_name     = "video-service"
 environment  = "production"
 instance_type = "t3.large"
 subnet_id    = var.private_subnet_id
}
```

Module lopala:

```
variable "app_name" {
 type = string
}

variable "environment" {
 type = string
}

variable "instance_type" {
 type = string
}

variable "subnet_id" {
 type = string
}
```

🔥 **Ippudu Pending Topics Complete Flow**

```
Terraform Core
    ↓
Terraform Advanced
    ↓
Production / Senior Level
    │
    ├── State Management
    ├── Drift Detection
    ├── Policy as Code
    ├── Security Scanning
    ├── Cost Management
    ├── Terraform Testing
    ├── Advanced Modules
    ├── Terragrunt
    └── OpenTofu
```

Nee existing 1–20 + advanced 1–9 already unna topics ki paina 10 production topics add chesthe, Terraform learning path chaala comprehensive avutundi.